

The Gaming Room

CS 230 Project Software Design Template

Version 1.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 4
Recommendations 5

Document Revision History

Version	Date	Author	Comments
1.0	09/20/26	Luis Felipe Jimenez Ferreira	Added executive summary, and requirements
1.1	10/04/2026	Luis Felipe Jimenez Ferreira	Added the development requirements for each Operating platform.
1.2	10/18/2026	Luis Felipe Jimenez Ferreira	Added the recommended Operating platform.

Instructions
Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

We are developing a web-based system similar to the 1980s game Win, Lose or Draw, but we are calling it Draw It Or Lose It. We will have one instance of the game running in the browser; we are saving the team with a unique name and Identification number(ID). Conceptually, we are saving each player's name, and we are assigning an ID to them. Each of the entities of the game, such as Team, Player, or Game, must be unique.

Requirements

Business Requirement: Accommodate as much processing power and RAM as the number of teams and players grows.

Technical Requirement: As the number of teams grows and we store all that data, probably in memory, it could max out, so we will need to allocate RAM based on how many teams and players we have.
*Technical Requirement: T*he system must ensure that only one game is running at any given time.
Technical Requirement: Every game and team instance must have a unique Identification.

Design Constraints

Design Constraint: To ensure that we have only one game instance at a time, we must implement the Singleton Pattern to ensure we only have a game instance running.
Design Constraint: Entity Identification & Unique ID generation.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

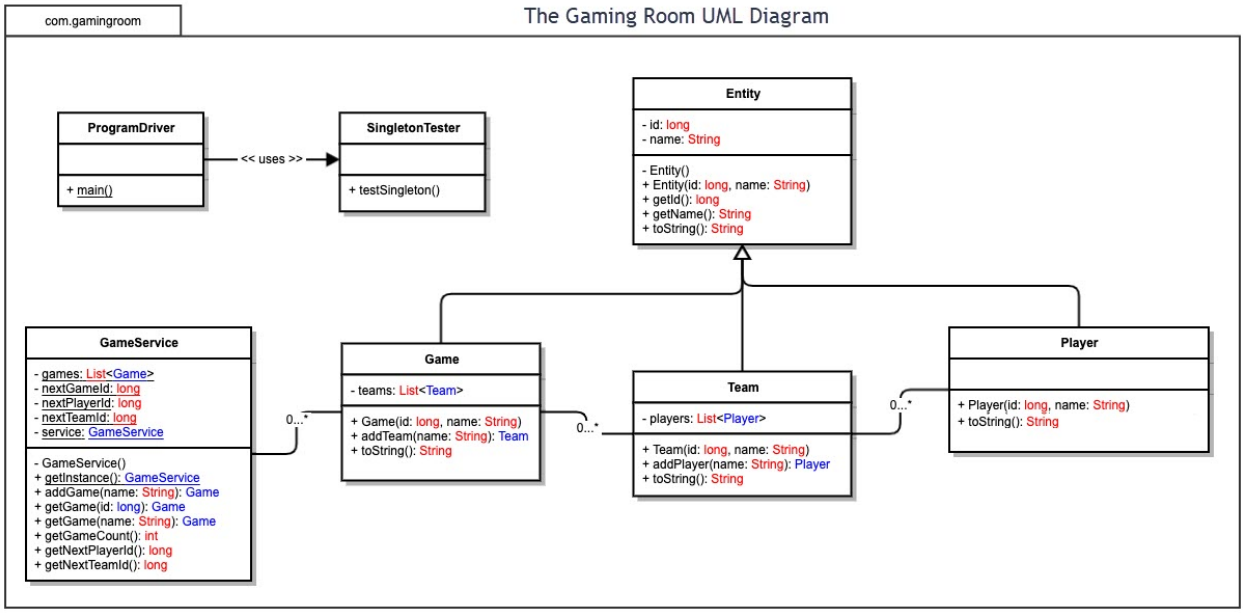
This UML Diagram puts into perspective all of the OOP principles. As I will describe each one below.

Inheritance: We see that the classes Game, Team, and Player inherit methods from the Entity class, which they receive: getId(), getName(), and toString(); the last one is overridden by the implementation.

Abstraction: We are setting the teams and players private so that only this class has access to them.

Encapsulation: We are protecting the list of teams and list of players from direct access, and there is currently no method to retrieve them.
Polymorphism: We have Game, Team, and Player; all are a form of Entity, and the GameService class has access to get its name and ID.

We also see that there is a strong relationship between the Player and a team and a team to a game, and we can have unlimited games, as the resources(space in memory, processing power, and graphics) can accommodate all running on the GameService class. Outside of this Relationship between classes, we have the ProgramDriver class that runs the main method, where all of this logic is being tested with the SingletonTester.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	**Unsuitable for enterprise web application hosting** It doesn't have a server-side platform such as Windows Server or Linux distributions like Ubuntu Server.	**Ideal and recommended for backend hosting.** Linux offers robust enterprise-grade server intrastructure platforms (Ex: Ubuntu server, Debian) and it supports Apache as a web server. Advantage: extremely lightweight, as it doesn't use many resources(RAM and CPU) Weakness: Requires specialized expertise	**Windows is a viable option for web application hosting**, but with higher resource usage. It provides enterprise server deployment capabilities via Windows Server Offers seamless integration and a GUI-based interface.	It does not apply. You can't host a web server on a mobile device.
Client Side	Cost: Low licensing costs; for development, could hire a front-end developer for 25 dollars an hour. Time: Development time is fast for The Gaming Room. Development time can be quick Expertise: Requires frontend web development skills(HTML5/CSS3/ JAVASCRIPT).Must ensure cross-browser testing for Apple Safari, and Google Chrome	Cost: It doesn't have any deployment fees, just the development cost. Development Time is fast, as we build for multiple browsers; we need to check browser support. Expertise: Requires web development skills and basic knowledge of Linux.	Cost: It doesn't have any client-side fees. Development Time: As per the other Operating platforms is fast as we are developing for multiple browsers. Expertise: Front-end Developer (HTML5, CSS3, JavaScript).	Cost: Publishing for Apple & Android stores requires a fee of $99 per year for the Apple Store. $25 publishing fee for Android. Time: Moderate, as we adjust it to different screen sizes. Expertise in mobile-first design, responsive media queries, and mobile browser performance optimization; expertise in CSS3 frameworks such as Bootstrap, Tailwind, or Materialize CSS.
Development Tools	Front-end technologies: HTML5, CSS3, JavaScript. Tools: Visual Studio Code, Xcode. Web Browser: Safari, Google Chrome. Licensing: Xcode is free.	Technologies: HTML5, CSS3, JavaScript Development Tools: Visual Studio Code or Visual. Web Browsers: Google Chrome, Mozilla Firefox Licensing: All development tools are free.	Front-end Technology: HTML5,CSS3. Programming Language: JavaScript. Tools: Visual Studio Code. Web Browsers: Google Chrome, Microsoft Edge, Mozilla Firefox	Technologies: HTML5, CSS3, JavaScript, and one of the following CSS Libraries: Bootstrap, Tailwind, Materialize. Tools: Visual Studio Code, Xcode Web Browsers: Google Chrome, Microsoft Edge, Mozilla Firefox, Safari Licensing: Android Studio and Xcode are free.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**: I recommend Linux over other Operating platforms because you don't have to update it as consistently as Windows. And over Apple, as it doesn't have support for servers. In today's market, most enterprise software runs on Linux because it doesn't use as much RAM or processing power. As I described before, the only downside of Linux is that it requires a user with specialized expertise in it, but the Ubuntu distribution is a beginner-friendly Linux Distribution.

2. **Operating Systems Architectures**: The Linux architecture is composed of the hardware, which includes RAM, CPU, Hard Disk Drive, Motherboard, and so on; then, on top of that, it's the Linux Kernel, which is built on the C programming language and is programmed to speak to the hardware itself, and the components communicate with each other with a low-level programming language such as C, which makes it very fast and lightweight. Then you have the shell, which is the terminal; it can be used to establish network connections, create files, remove files, write to files anything that you can do on an operating system, you can do on the Shell. Lastly, you have the applications; these could be written by you or third parties using a compatible programming language for that system. Applications can provide a graphical user interface, which makes it more intuitive for a general audience.

3. **Storage Management**: The book discusses the **Ext**ended **F**ile **S**ystem (**EXTFS**), which was restricted to 14-character file names and a maximum file-system size of 64 MB, but it has had multiple versions. Now, the latest is EXT4FS, which is the fourth version of this file system. I recommend this version because it focuses on scalability and reliability. Along with the RAID 10 (1+0) configuration, it is a strong option for transferring and receiving data, which is server-client communication.

4. **Memory Management**: Linux separates physical memory into three different zones or regions: ZONE_DMA (0<16mb) this zone can only be accesible for DMA devices such as disk drive controllers, graphics cards, network cards, sound cards and dedicated **DMA** controllers, ZONE_NORMAL (16.. 896MB); this zone is mapped to the CPU's address space and its used for most routine memory requests; and ZONE_HIGHMEM (> 896) this last one is reserved for memory that is not mapped into the kernel space and it's used for user processes. The high memory zone is where Draw IT or Lose It will be used to allocate memory for processes to load the images, users, teams, and handle input requests.

5. **Distributed Systems and Networks**: According to the book Operating Systems Concepts, Section 21.10, networking is a key area of functionality for Linux; it has internet protocols for most UNIX-to-UNIX communication and also non-UNIX, meaning that for a distributed system in a network it has protocols to establish connections from a variety of protocols such as TCP(Transmission Control Protocol)/IP(Internet Protocol) /UDP(User Datagram Protocol)/FTP(File Transfer Protocol)/ SSH(Remote login protocol)/ SMTP(Email Protocol). Draw It or Lose It, since it's a web-browser application it will connects via TCP/IP with its users. But for other operations, such as internal file transfer, it can use FTP for File transfer. SSH for remote system access. Something to consider is that if a Node(Computer) of the network doesn't have these protocols set up, it will deny access; it has a firewall management system for when these events occur.

6. **Security**: Linux's security model is closely related to UNIX security mechanisms. First, it has Authentication that ensures nobody can access the system without proving they have entry rights. Access control provides mechanisms for checking if a user has the right access to a certain object. More custom security measures can be added at the application layer to fit Draw It or Lose It requirements.